TAPYRA

Documentação do Projeto

Rede de Alerta de Fumaça e Fogo: monitor de queimadas na Amazônia

Projeto de Frontend · Outubro de 2026

1. Visão geral

O **Tapyra** é uma aplicação web que monitora queimadas na Amazônia quase em tempo real. Ele usa os **focos de calor detectados por satélite** (API NASA FIRMS) e os transforma em informação simples: onde há fogo, qual é a situação perto de Manaus e quando existe um alerta.

Objetivo: transformar dados técnicos em avisos úteis para moradores, escolas, unidades de saúde e comunidades da Amazônia.

Problema que resolve: a informação sobre fogo e fumaça existe, mas é técnica e está espalhada em vários lugares. O Tapyra reúne tudo em uma interface clara, que qualquer pessoa entende.

O que o sistema oferece

- **Mapa interativo** com cada foco de calor desenhado como um círculo, com zoom, duas camadas de visualização e balão de informações ao clicar.
- **Cards de resumo** na página inicial, atualizados conforme a área que o usuário está vendo no mapa.
- **Página de alertas** com avisos gerados a partir dos focos reais, classificados por nível (Atenção, Alerta, Crítico).
- **Página Sobre**, que explica o projeto, as fontes de dados e os contatos de emergência (Bombeiros 193 e Defesa Civil 199).

2. Tecnologias utilizadas

Tecnologia	Versão	Para que serve no projeto
React	19	Base da interface, construída com componentes reutilizáveis.
Vite	8	Ferramenta que roda o projeto em desenvolvimento e gera a versão final, de forma muito rápida.
React Router DOM	7	Navegação entre as páginas sem recarregar o site (SPA).
Bootstrap + React-Bootstrap	5.3 / 2.10	Layout responsivo e componentes prontos: Container, Row, Col, Card, Badge, Navbar, ListGroup.
Leaflet + React-Leaflet	1.9 / 5	Mapa interativo gratuito e de código aberto.

Tecnologia	Versão	Para que serve no projeto
API NASA FIRMS	-	Fonte dos focos de calor (satélite VIIRS).
ESLint	10	Verifica o código e aponta erros e más práticas.
Git + GitHub	-	Controle de versão, branches e Pull Requests em equipe.
Vercel	-	Publicação (deploy) automática do site.

3. Como executar o projeto

1. Instale o **Node.js 20 ou 22** (versões LTS). Com o Node 18, o Vite não inicia.
2. Clone o repositório e entre na pasta do projeto.
3. Instale as dependências: `npm install`
4. Crie na raiz do projeto um arquivo **.env** com a chave gratuita da NASA FIRMS:
   `VITE_FIRMS_MAP_KEY=sua_chave`
5. Inicie o servidor de desenvolvimento: `npm run dev` e abra `http://localhost:5173`.
6. Para verificar a qualidade do código: `npm run lint`. Para gerar a versão final: `npm run build`.

> O arquivo **.env** não é enviado ao GitHub (está no .gitignore). Cada pessoa da equipe cria o seu.

4. Estrutura de pastas

Caminho	Responsabilidade
src/main.jsx	Ponto de entrada: monta o React na página.
src/App.jsx	Define as rotas, a barra de navegação, o rodapé e o provedor de dados.
src/pages/	As 4 telas: Home, MapPage, Alerts e About.
src/components/	Peças reutilizáveis: Navbar, Footer, StatusCard, NewsCard e MapView.
src/hooks/	Lógica reaproveitável em hooks: useHomeStats.
src/context/	FireDataContext: compartilha os focos entre o mapa e os cards.
src/services/	firms.js: comunicação com a API da NASA e conversão dos dados.
src/App.css e index.css	Estilos globais e variáveis de cores.
src/documentation/	Documentação em PDF do projeto.

5. Rotas (React Router)

As rotas ficam em um único arquivo, o `App.jsx`. A barra de navegação (com `NavLink`, que destaca a página ativa) e o rodapé ficam fora das rotas, por isso aparecem em todas as telas. Só o conteúdo central troca.

Rota	Componente	Conteúdo
/	Home	Apresentação, cards de resumo e mapa reduzido com botão para o mapa completo.
/mapa	MapPage	Mapa em tamanho completo com os focos de calor.
/alertas	Alerts	Lista de alertas por nível e legenda de como os níveis funcionam.
/sobre	About	Objetivo, fontes de dados e informações do projeto.

6. Fluxo dos dados

Esta é a ideia central do sistema: do satélite até o que o usuário vê na tela.

1. O usuário abre ou move o mapa.
2. O **MapView** detecta o movimento (evento `moveend` do Leaflet), espera um instante (debounce) e cancela buscas antigas com `AbortController`.
3. O **firms.js** pede à NASA os focos da área visível. A resposta vem em **CSV**.
4. O CSV é convertido em uma lista de objetos (latitude, longitude e demais campos).
5. O MapView desenha cada foco como um `CircleMarker` e publica a lista no **FireDataContext**.
6. Os cards da Home leem o contexto (pelo hook `useFireDataContext`) e mostram os números atualizados.
7. A página de **Alertas** faz a própria busca, gera os alertas a partir dos focos e se atualiza a cada 5 minutos.

7. Regras de lógica

Conversão do CSV

A NASA devolve um texto com colunas separadas por vírgula. O `firms.js` divide em linhas e colunas e monta objetos que o mapa sabe desenhar. Também detecta mensagens de erro da API para avisar o usuário.

Cálculo de distância (fórmula de Haversine)

Para saber se um foco está perto de Manaus, calculamos a distância entre dois pontos de latitude e longitude considerando a curvatura da Terra. Os cards da Home contam focos em um raio de **700 km** de Manaus.

Regras de geração de alertas

Usam os focos das últimas 24 horas (satélite VIIRS) na área de cerca de 700 km ao redor de Manaus. A página atualiza sozinha a cada 5 minutos.

Condição	Nível	Mensagem exibida
1 a 9 focos a até 50 km de Manaus	Atenção	Quantidade de focos e distância do mais próximo.
10 ou mais focos a até 50 km de Manaus	Alerta	Mesma mensagem, com nível elevado.
Focos entre 50 e 150 km	Atenção	Focos na região metropolitana e municípios vizinhos.
30 ou mais focos na área monitorada	Atenção	Concentração de focos no Amazonas.

Níveis previstos na legenda da página: Atenção (focos novos num raio de 50 km), Alerta (atenção somada a aumento de PM2.5 ou notícia confirmada) e Crítico (PM2.5 em faixa ruim por tempo prolongado). Os dados de qualidade do ar ainda não foram integrados, por isso hoje só os alertas por proximidade de focos são gerados.

8. Layout e experiência do usuário

A identidade visual segue o tema do projeto: o **verde da floresta** e o **laranja do fogo**. As cores ficam em variáveis CSS, então uma alteração vale para o site inteiro.

Cor	Variável	Hex	Uso
	--verde-escuro	#0d3b2e	Cabeçalho, rodapé e títulos
	--verde-medio	#1a5c45	Elementos de apoio
	--verde-claro	#2d8a6a	Detalhes e botões secundários
	--laranja	#e67e22	Destaques e botões principais
	--vermelho-alerta	#c0392b	Gravidade alta
	--amarelo-atencao	#f1c40f	Nível de atenção

Decisões de UX

- **Cores com significado universal:** amarelo, laranja e vermelho comunicam gravidade crescente sem precisar ler.
- **Hierarquia clara:** na Home vêm primeiro os números, depois o mapa, com um botão para abri-lo em tela cheia.
- **Responsividade:** grid do Bootstrap (Container, Row e Col) reorganiza o conteúdo em celulares e tablets.
- **Feedback ao usuário:** mensagens de carregando e de erro, para a pessoa nunca ficar sem resposta.
- **Segurança:** o rodapé traz os telefones de emergência (193 e 199) e o aviso de que o sistema complementa os órgãos oficiais.
- **Mapa explorável:** zoom, troca de camada base e balões com informações de cada foco.

9. Trabalho em equipe no GitHub

Branch	Função
main	Versão estável. Protegida: não aceita push direto, só Pull Request.
develop	Integração: reúne o trabalho de todos antes de ir para a main.
feat/joaod, Feat/Walisson, Feat/Guilherme	Branches individuais de cada integrante, uma por funcionalidade.

- O código é reunido por **Pull Requests** (do #2 ao #9 no histórico).
- Conflitos de merge entre branches foram resolvidos em equipe.
- A Vercel comenta em cada PR com o resultado do deploy, funcionando como uma verificação automática.
- **Lição aprendida:** um PR foi mesclado na main em vez da develop. A correção foi feita com `git revert` por meio de um novo PR (#9), sem apagar histórico e sem perder o trabalho. A

partir daí, conferimos sempre a branch de destino antes de abrir um PR.

10. Onde cada critério de avaliação aparece

Critério	Como aparece no projeto
Componentização	Navbar, Footer, StatusCard (usado 4 vezes), NewsCard e MapView (usado na Home e no /mapa).
Router	BrowserRouter, Routes e Route em App.jsx; NavLink na barra de navegação.
Hooks	useState, useEffect, useCallback, useMemo, useRef, createContext, useContext, useMap, useMapEvents e hooks próprios (useHomeStats, useFireDataContext).
Layout (UI/UX)	Paleta em variáveis CSS, Bootstrap responsivo, mapa interativo, cores de alerta e rodapé de emergência.
Lógica	Conversão de CSV, busca por área visível, fórmula de Haversine e regras de geração de alertas.
Trabalho em equipe	Branches por pessoa, develop, main protegida, Pull Requests e resolução de conflitos.
Documentação	Este documento, README, PDF em src/documentation e histórico do GitHub.
Biblioteca	React, Vite, React Router, Bootstrap, React-Bootstrap, Leaflet, React-Leaflet e ESLint.
Consumo de API	NASA FIRMS via fetch, chave em .env, proxy no Vite, loading, erro e AbortController.
Deploy	Projeto conectado à Vercel, com publicação automática a cada atualização.

11. Limitações atuais e próximos passos

- **Alertas Alerta e Crítico completos:** dependem de dados de PM2.5, vento e notícias, ainda não integrados.
- **Card de notícias:** ainda é de demonstração.
- **Regra única:** a Home e a página de Alertas calculam seus números separadamente. O ideal é centralizar a regra em um só lugar.
- **Texto da Home:** menciona 300 km, mas o código usa 700 km de raio. É necessário alinhar o texto ao valor real.
- **Chave da API no navegador:** variáveis VITE_* ficam visíveis no site publicado. O ideal é uma função de servidor intermediária.
- **Deploy:** em produção, o proxy do Vite não existe. É preciso garantir a chave nas variáveis de ambiente da Vercel e a chamada à NASA.
- **Qualidade de código:** remover o hook duplicado sem uso (useFireData.js) e ajustar os avisos do ESLint.